



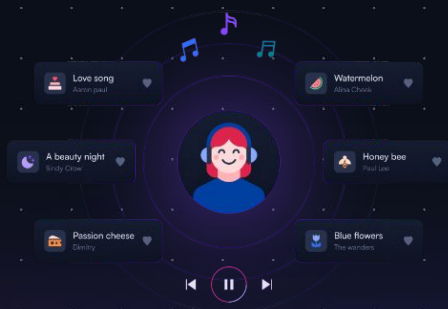
TurboQuant in Qdrant: a technical deep dive.

Chadha Sridi - DevRel @ Qdrant

Vector search application spectrum



Search Systems



Recommendations



Anomaly Detection



RAG & Agents

One engine behind every workload

MULTIMODAL

Tripadvisor

2-3× more revenue from
AI-engaged users · 1B+
items

RECOMMENDATIONS + RAG

HubSpot

Breeze AI – chosen after
it outperformed every
alternative on retrieval

SEARCH AT SCALE

Bazaarvoice

2.7B vectors · ~100×
smaller footprint · <100
ms @ ~98% recall

HEALTHCARE

Roche

SEMANTIC FILTERING

OpenTable

AI Concierge – sparse
embeddings filter 60K+
restaurants

REAL-TIME MULTIMODAL

Flipkart

9 h → <1 min fraud-
detection turnaround on
live retrieval

AGENTS · KNOWLEDGE

Dust

5,000+ data sources ·
multi-second → sub-
second · ¼ the RAM

AGENTS

Lyzr

>90% latency cut · 300-
500 ms → 20-50 ms

PLATFORM SCALE

Canva

Standardizing vector
search across AI
features · millions of
users

MANUFACTURING

Bosch

Video anomaly detection example



Qdrant
Edge

Sentinel

Video anomaly detection
Twelve Labs + Qdrant

Cameras

Incidents

Search

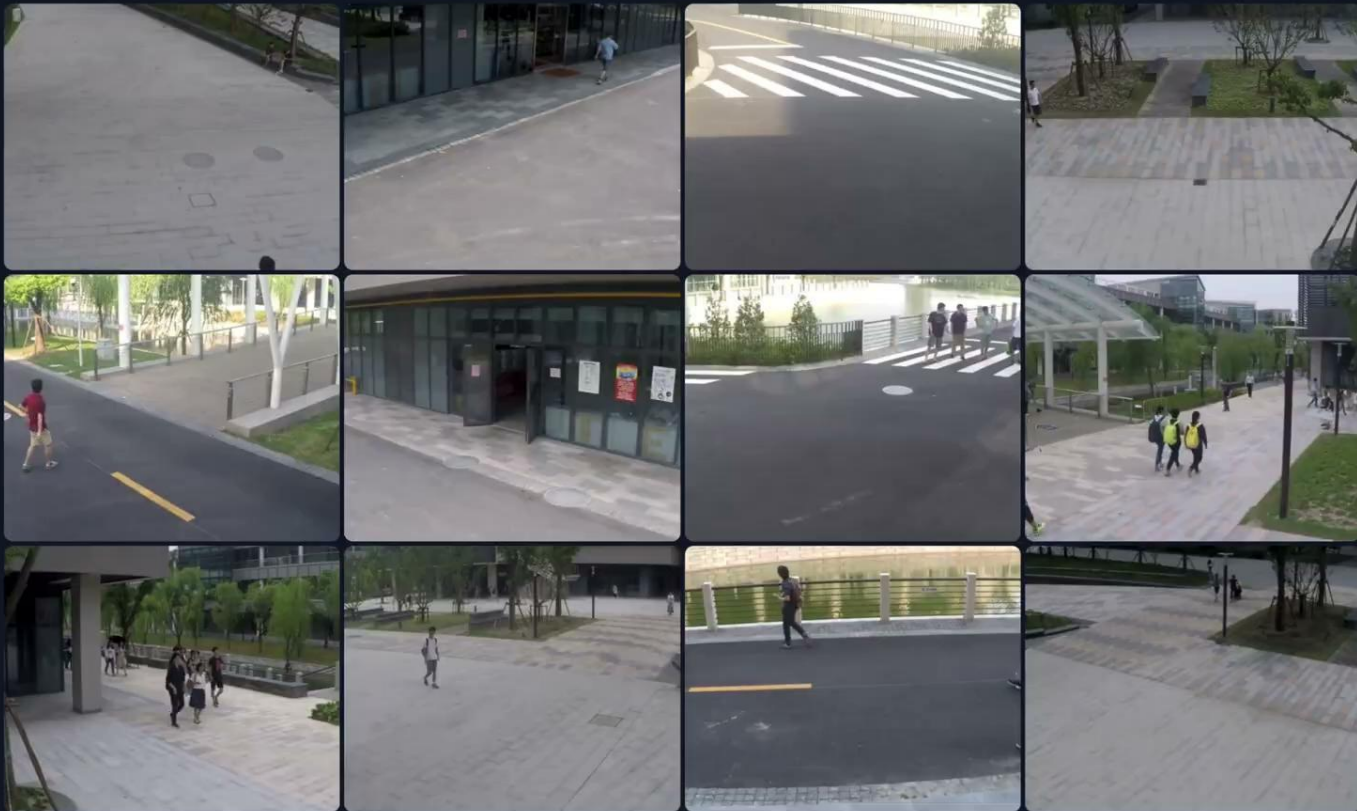
Chat

Intelligence



Take a tour

Search incidents...



Campus Alerts

61 new

12 cameras · 3 zones · 61 alerts

☐ Hide reviewed

1 99 Five people engage in physical fight

Main Entrance · Zone A

Fighting 2:12:04 PM · 20s

Paved forecourt, multiple approach lines

2 98 Two individuals fight on crosswalk

Library Plaza · Zone A

Fighting 2:11:36 PM · 13s

Roadway with zebra crossing

3 97 Group fight erupts while two people ru...

North Path · Zone B

Fighting 2:12:21 PM · 24s

Curving roadway with crosswalk

4 94 Multiple people cross the pedestrian a...

Library Plaza · Zone A

Running 2:11:48 PM · 15s

Roadway with zebra crossing

5 94 Four people run quickly across crossw...

Library Plaza · Zone A

Running 2:13:32 PM · 10s

Roadway with zebra crossing

6 89 Ten people walk along sidewalk, includ...

East Gate · Zone A

Vehicle 2:11:31 PM · 14s

Glass façade, sidewalk and curb lane

Man exits with phone, cyclist passes b...

POWERED BY



Qdrant Edge



Twelve Labs



Vultr



NVIDIA

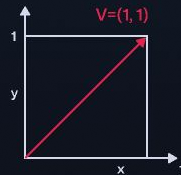
Vector search

Embeddings capture semantics.

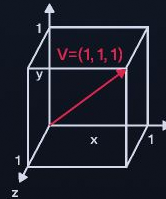
Can represent any kind of data.

Similar objects have close embeddings.

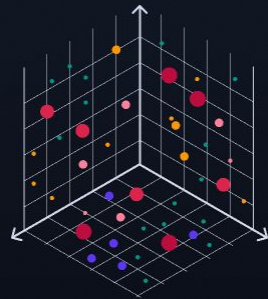
Qdrant handles embeddings natively.



2-Dimensional
Vector



3-Dimensional
Vector



Embedding
Model



[0.21, -0.32, 0.68, ...]

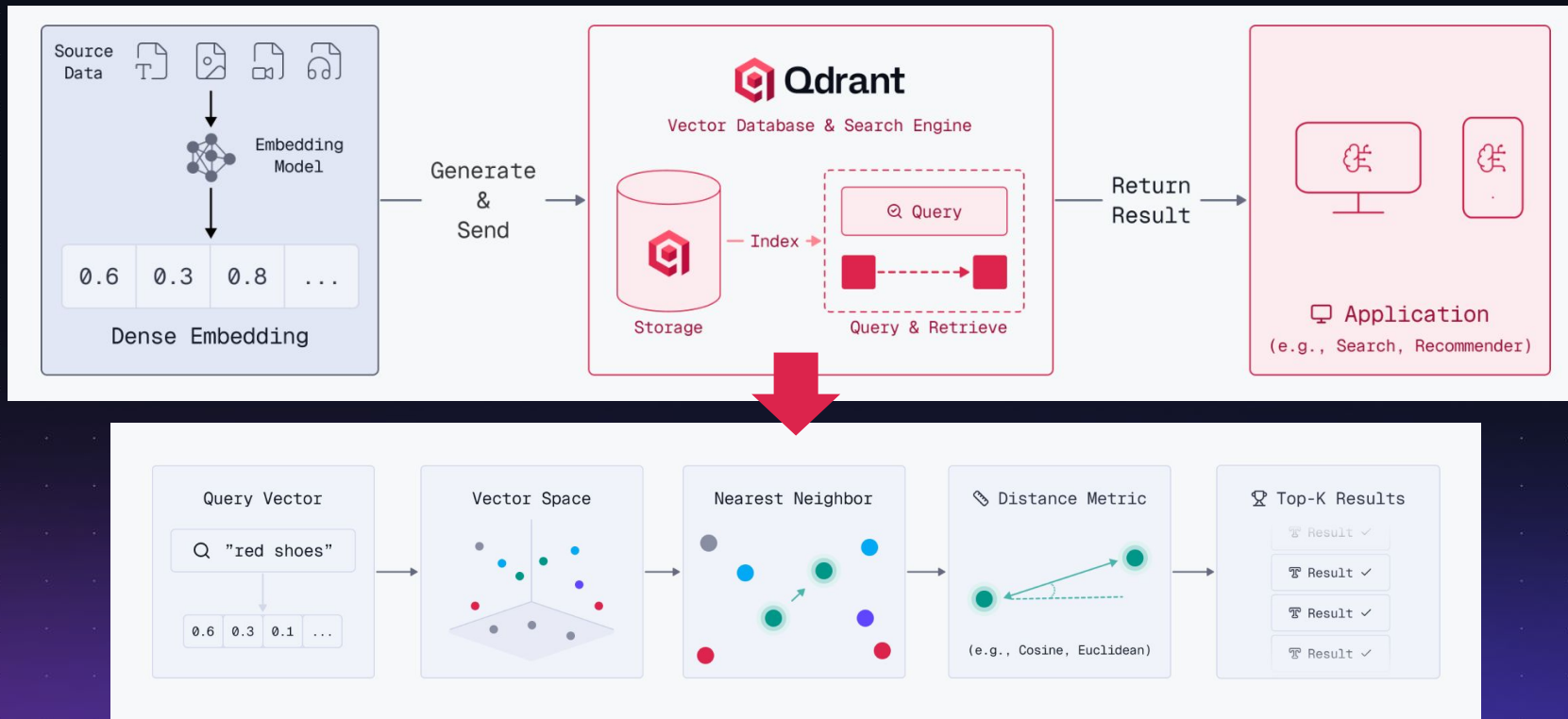
[0.79, -0.14, 0.83, ...]

[-0.94, -0.42, 0.69, ...]

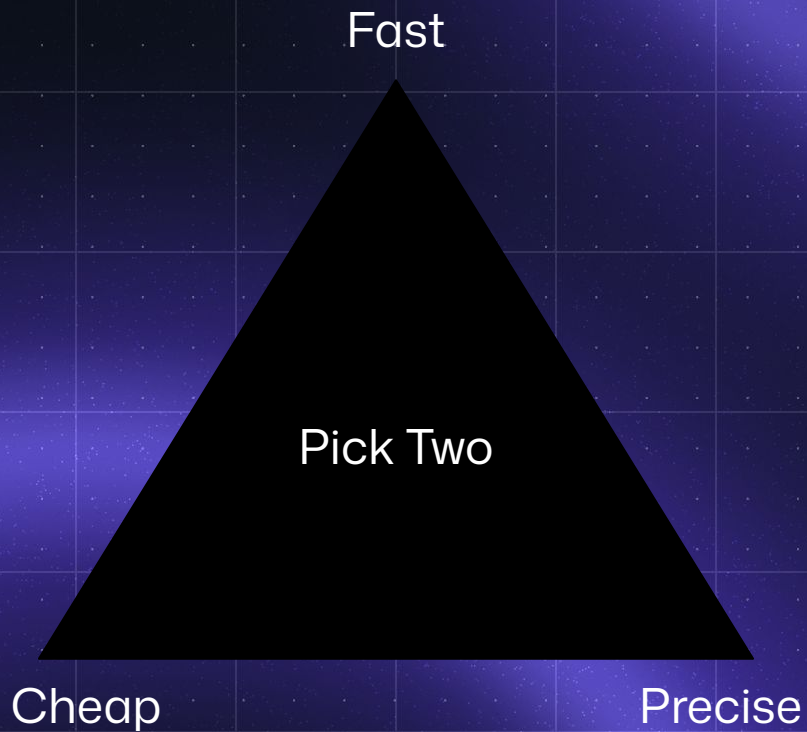
Data Objects

Vector Embeddings

The search workflow



The tradeoff you have to make



Composable retrieval primitives

Tune retrieval to your needs



Filtering

Numeric, categorical, geo, temporal filters out-of-the-box



Vector Search

Scalable similarity and discovery search (billions of vectors)



Hybrid Search

Combine dense + sparse embeddings, filters, and metadata



Multi-vectors

Late interaction for retrieval models (e.g. ColBERT)



Re-ranking

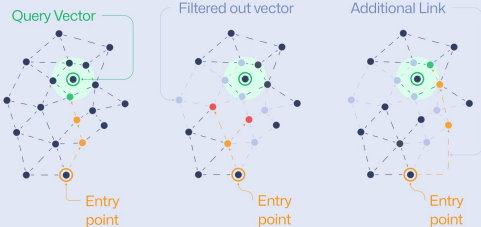
Maximum Marginal Relevance (MMR), score boosting



Quantization

Binary, scalar, product, TurboQuant; lower cost without major recall loss

Filterable HNSW



Similarity Search



Similarity Search with MMR



A quick note on deployments



- **Qdrant Open Source**

Self-host with Docker.

- **Qdrant Managed Cloud**

Fully managed (management UI and API).

- **Qdrant Hybrid Cloud**

Use Qdrant Cloud's UI to create and manage your clusters, while they remain within your infrastructure.

- **Qdrant Private Cloud**

Fully on-premise. Supreme data privacy and sovereignty for regulated industries.

- **Qdrant Edge**

For constrained devices

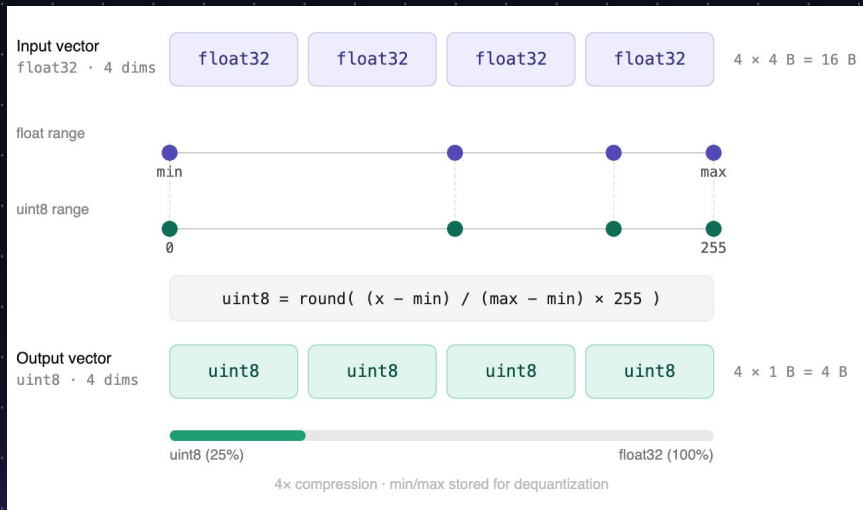
Quantization types in Qdrant

Scalar Quantization (INT8):

- Float32 → 8-bit integers: 4x compression
- Map values to 0-255 range
- Typically < 1% accuracy loss
- ~2x faster brute-force search

Binary Quantization:

- Each dimension → 1 bit: 32x compression
- 512 GB → 16 GB for 1M documents
- Use rescoreing to recover accuracy



Product Quantization

enables up to 64x compression
when minimizing memory is the top
priority.

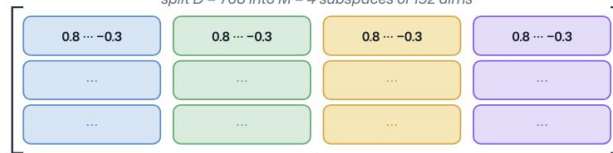
Product Quantization

split each vector into sub-vectors – quantize every subspace with its own k-means codebook

split $D = 768$ into $M = 4$ subspaces of 192 dims

Set of vectors

$N \times 768$



sub-vector 1

$\mathbb{R}^{192}$

sub-vector 2

$\mathbb{R}^{192}$

sub-vector 3

$\mathbb{R}^{192}$

sub-vector 4

$\mathbb{R}^{192}$

Cluster each subspace independently with k-means

◆ = centroid (codebook entry)



subspace 1

k-means · k = 256



subspace 2

k-means · k = 256



subspace 3

k-means · k = 256



subspace 4

k-means · k = 256

each subspace learns its own codebook $C_m = \{c_0 \dots c_{255}\}$, $c_i \in \mathbb{R}^{192}$

$\mathbf{x}$
4 sub-vectors



quantized
indices · 4 bytes



768x
3072 B → 4 B

$\hat{\mathbf{x}}$
≈ x · reconstructed



TurboQuant X Qdrant

- Shipped in Qdrant 1.18
- Introduced by Google Research

Paper highlight:

- Compresses float32 vectors by 8× with virtually no loss in NN search quality.
- Requires no per-dataset codebook training.

Qdrant implementation:
Adds extensions that make it work on **real production** embeddings.

TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate

Amir Zandieh
Google Research
zandieh@google.com

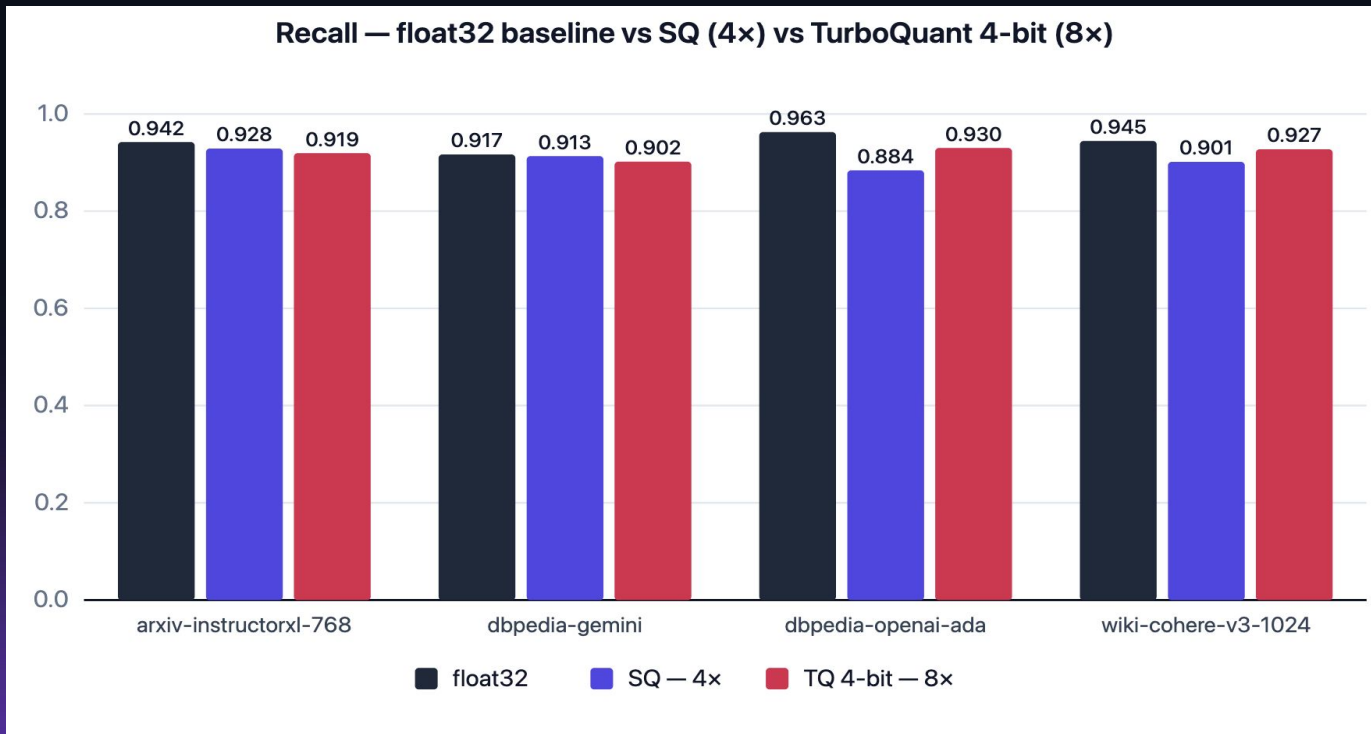
Majid Daliri
New York University
daliri.majid@nyu.edu

Majid Hadian
Google DeepMind
majidh@google.com

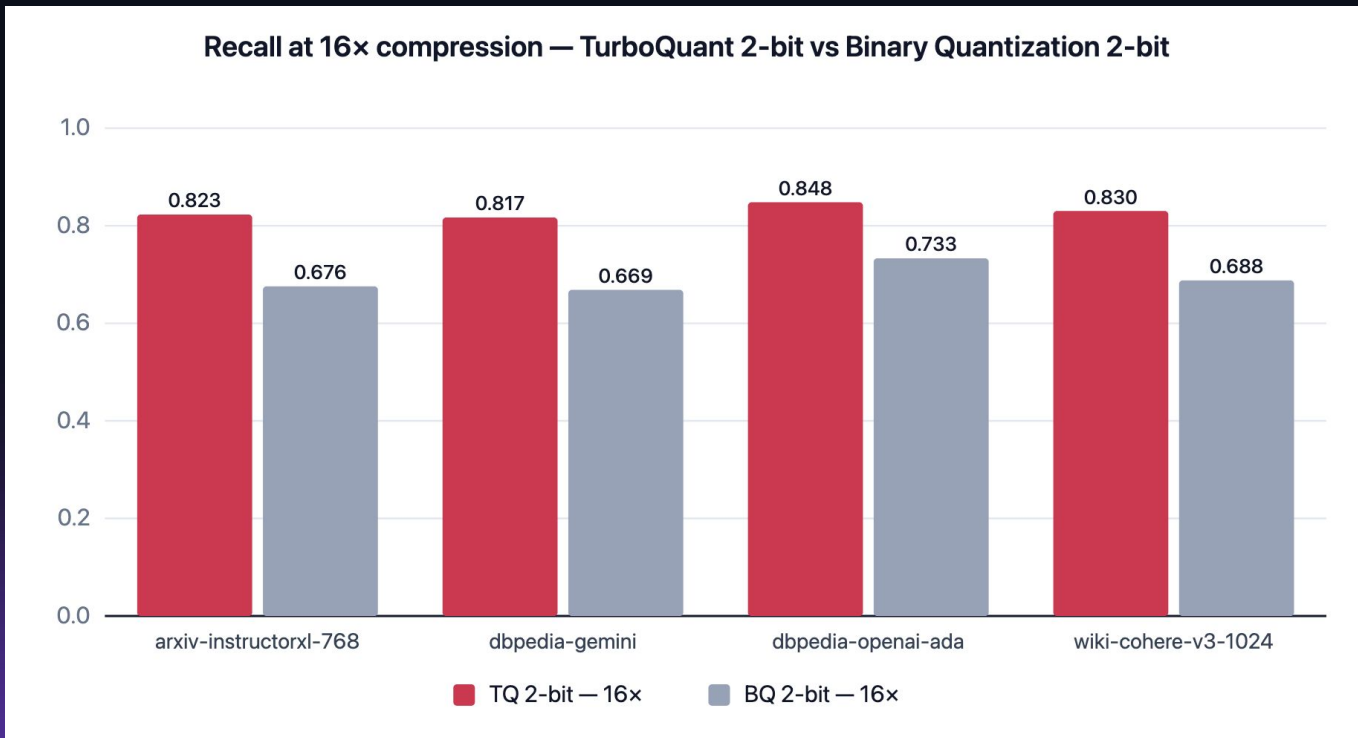
Vahab Mirrokni
Google Research
mirrokni@google.com

Benchmarks

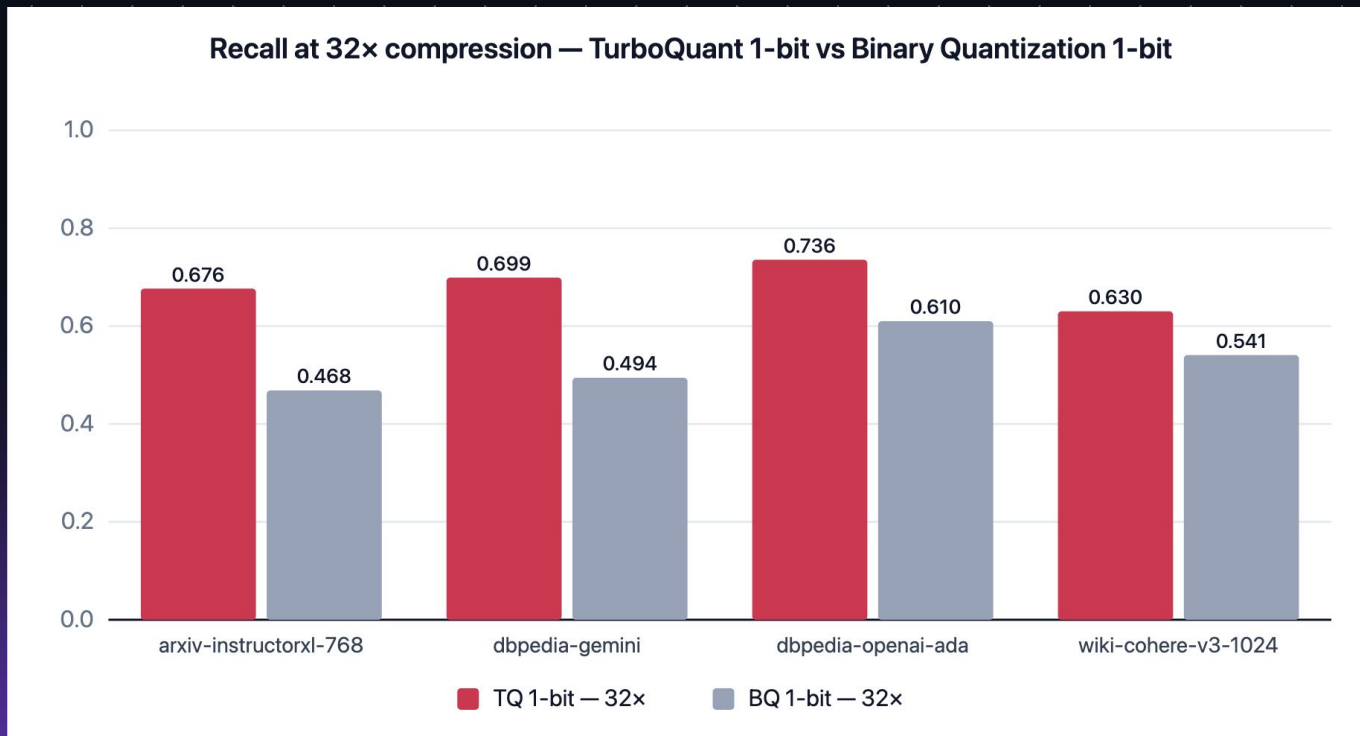
TQ 4-bit is competitive with SQ at half the storage. On arxiv-instructorxl and dbpedia-gemini it is about 1 pp below SQ. On dbpedia-openai-ada and wiki-cohere-v3 it actually *beats* SQ by up to 4.6 pp.



TQ 2-bit beats BQ 2-bit by 11-15 pp on these four datasets (and 9-24 pp across all ten datasets), at the same 16x storage.



TQ 1-bit beats vanilla BQ 1-bit by 9–21 pp on these four datasets (and 9–21 pp across all ten datasets), at the same 32x storage.



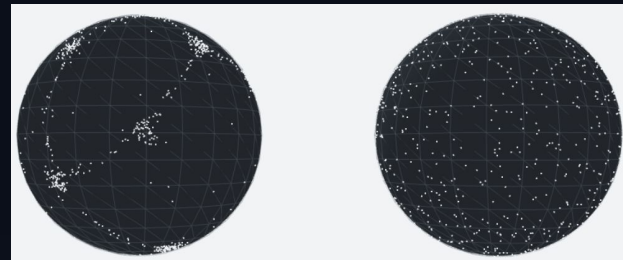
The idea:

Don't fight your data's distribution – transform it into one we can quantize “perfectly”.

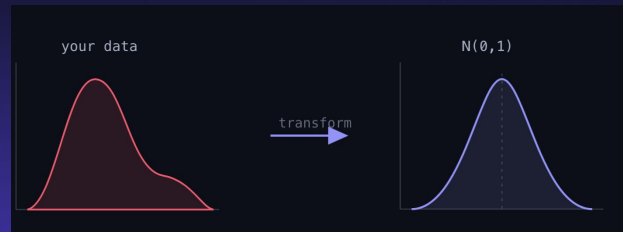
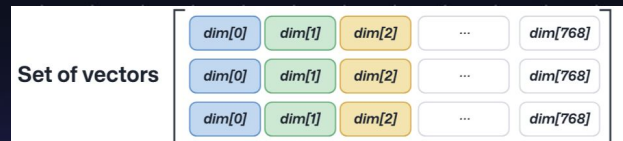
We know how to compute an optimal **Lloyd–Max** quantizer for data following a standard Gaussian distribution $N(0,1)$, so instead of trying to quantize our data as is, what if we try to transform it to this $N(0,1)$ distribution and use the optimal quantizer?

Theory vs Practice

Qdrant implementation makes tradeoffs (distortion for speed)



Anisotropic vs Isotropic data



Example of one coordinate distribution

What's cool about this

Data Agnostic

- The quantizer is data agnostic → no data specific k-means training per sub-vector space like in PQ.

Shared Codebook

- One codebook per bit-width budget shared across all the dimensions.
- **Known** and **hardcoded** into the code.

Near optimality

- The Lloyd-Max codebook for a Gaussian distribution at a given bit budget is the optimal scalar quantizer in terms of MSE

Max LLoyd optimal codebook

Given a bit-width budget (4-bit, 2-bit, 1-bit, 1.5-bit), map the float32 values of the vector to the **index** of the closest centroid

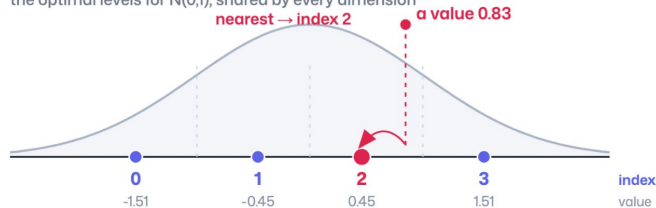
```
PUT /collections/{collection_name}
{
  "vectors": {
    "size": 1536,
    "distance": "Cosine"
  },
  "quantization_config": {
    "turbo": {
      "bits": "bits2",
      "always_ram": true
    }
  }
}
```

Quantization with a shared codebook

bit-width 2 $\rightarrow$ 4 centroids · every dimension is stored as one index

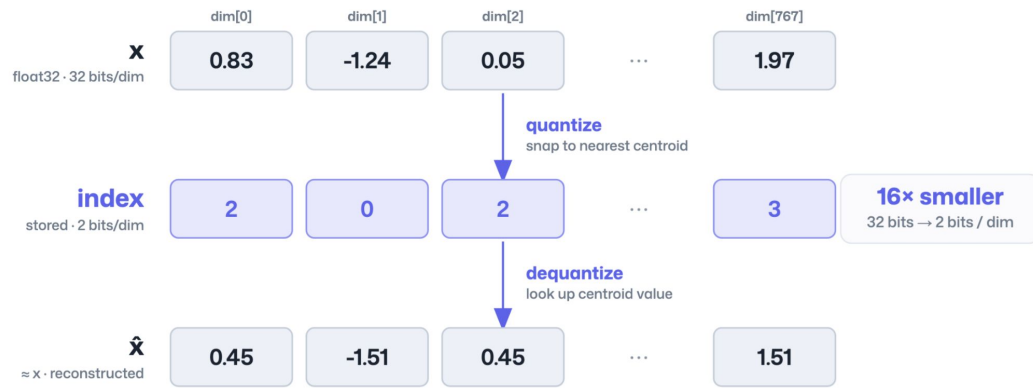
The codebook – 4 centroids

the optimal levels for $N(0,1)$, shared by every dimension



Lookup table

index	value
0	-1.51
1	-0.45
2	0.45
3	1.51



Step 1: Random rotation

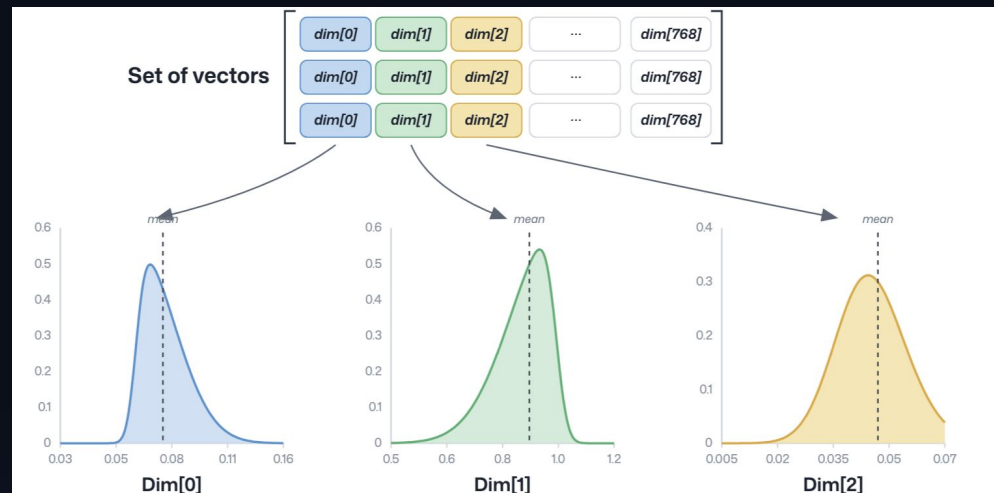
Goal: Spread the variance so all the coordinates carry the same energy - information level.

How: Multiplying the vector by a rotation matrix mixes the dimensions.

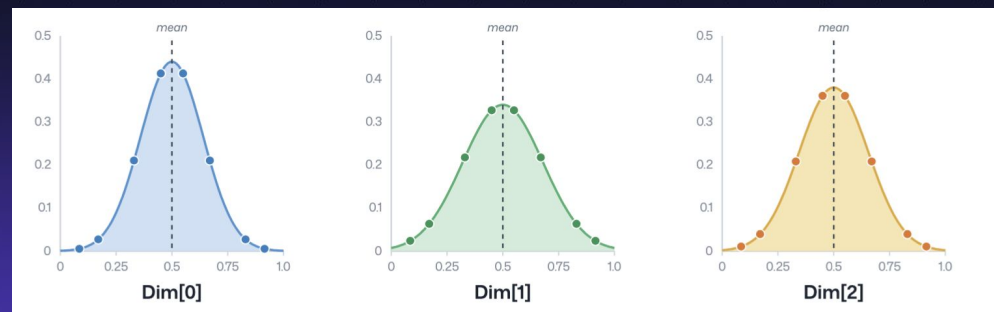
Example: $[5.0, 0.2, 0.1] \times \begin{bmatrix} 0.1 & \dots & \dots \\ \dots & \dots & \dots \\ \dots & -0.4 & \dots \end{bmatrix} = [0.9, 0.5, 0.4]$

Our vector Rotation matrix Rotated vector

- Each output coordinate is a sum of many inputs - in high dimensions, that sum is approximately Gaussian (CLT).
- Estimated via the **Hadamard Transform**. $O(n^2) \rightarrow O(n \log n)$
- Preserves L2 distances, dot products, cosine similarities.



↓ Random Rotation ↓



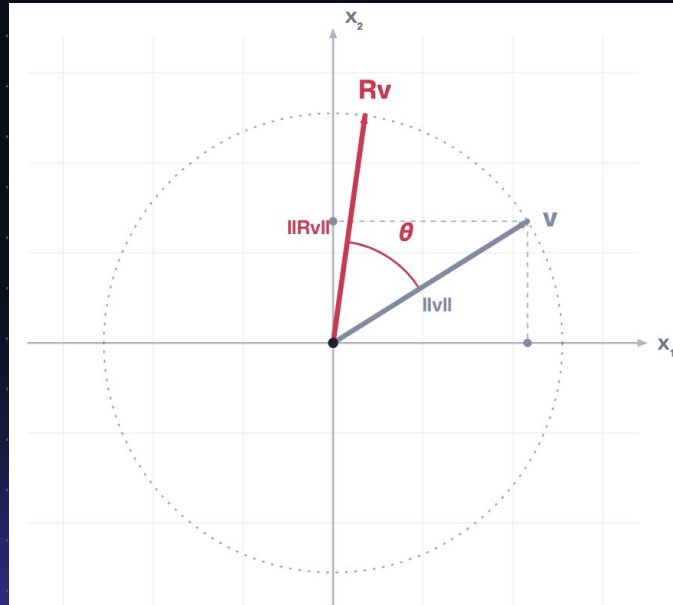
TL;DR: The Rotation trick

Changing the coordinate system, not the geometry

- Same distances, same neighbors.
- What matters for vector search: the similarity ranking.
- What *does* change: the statistical distribution of the coordinates → More amenable to near optimal quantization.

Note: This doesn't work for L1 distance.

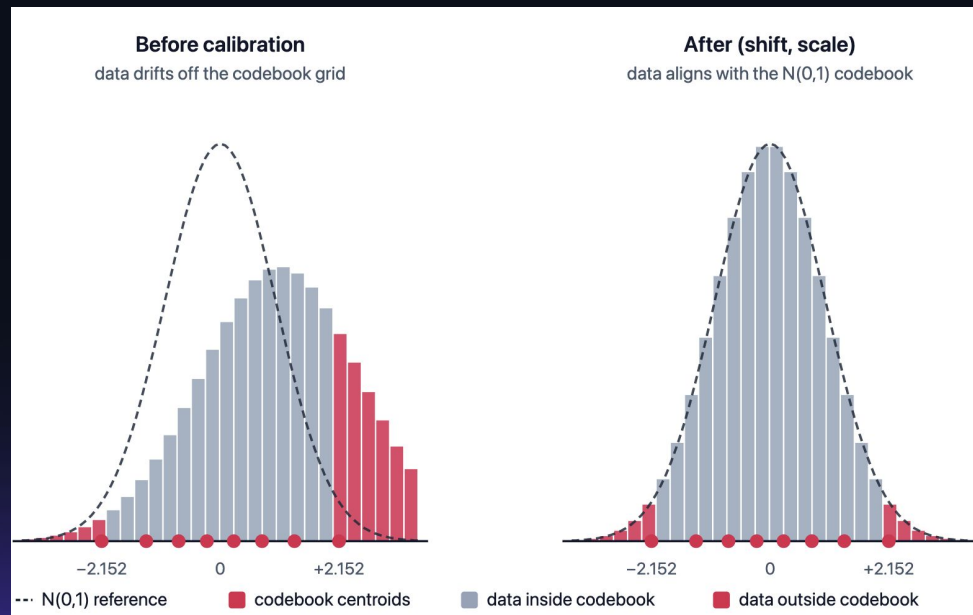
From this point on, everything is rotated: vectors in the collection and vector queries.



Rotation in 2D plane

Step 2: Per-Coordinate Calibration

- After the rotation step coordinate distributions are *Gaussian-ish*
- The Lloyd-Max codebook is fitted once for $N(0, 1)$.
- Coordinates that drift off the codebook grid waste centroid positions and lose recall.
- We **shift and scale** the coordinates' distributions to the codebook's grid.
- Save the shift and scale metadata per segment.
→ compensate for this during dot product computation between queries and vectors.



Step 3: Quantization

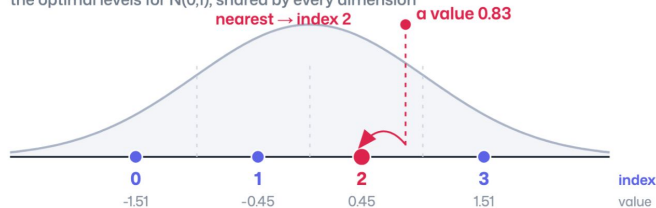
- Apply The hardcoded optimal LLoyd-Max codebook.
- Each float32 value is mapped to the index of its nearest centroid.

Quantization with a shared codebook

bit-width 2 $\rightarrow$ 4 centroids · every dimension is stored as one index

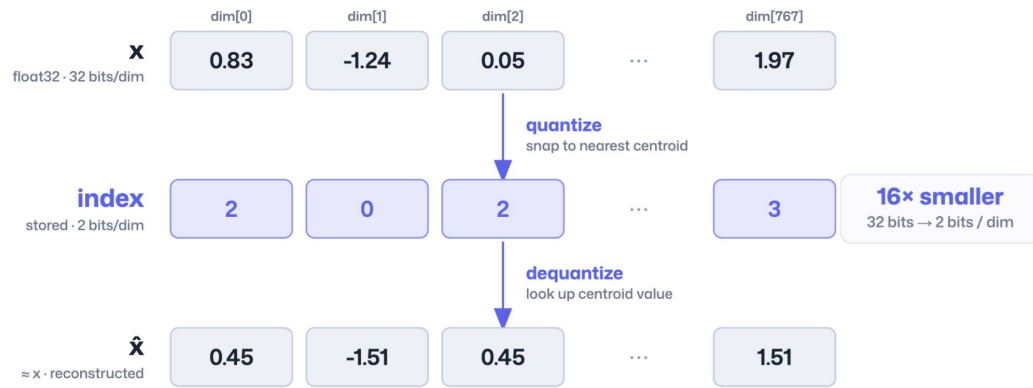
The codebook – 4 centroids

the optimal levels for $N(0,1)$, shared by every dimension



Lookup table

index	value
0	-1.51
1	-0.45
2	0.45
3	1.51

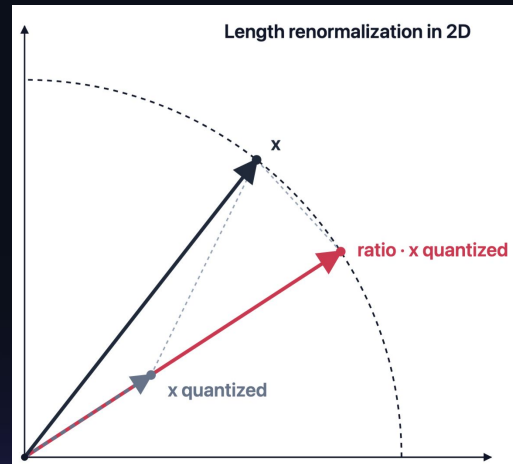


More Qdrant extensions

Length renormalization

- Borrowed from RaBitQ
- **Issue:** Norm of the dequantized vector is different from the norm of the original vector $\rightarrow$ distortion on the inner product.
- **Solution:** store one extra per-vector scalar that records the ratio of original length to centroid-reconstruction length.

SIMD acceleration



The full pipeline

Rotate

Hadamard transform.
spreads variance →
Gaussian-ish, distances
preserved.

Calibrate

Shift & scale to fit the
codebook's centroid grid.
Save metadata per segment

Quantize

Map each coordinate value
to the index of the nearest of
the 2^b near optimal levels

Store

Keep 8x/16x/32x
compressed vectors.

A practical guide

- You currently run SQ → try TQ 4-bit. Comparable recall at half the memory.
- You currently run BQ at any bit depth → try TQ at the same storage budget. It consistently delivers higher recall (10–20 pp) at both the 16x and 32x storage classes.
Stay on BQ if you observe a noticeable drop in throughput on your workload, or if the recall improvement is too small to matter for your use case.
- L1 → stay on SQ.

Thank You!



Chadha Sridi

DevRel Engineer